Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Dersin yapısı
  - Bu ders öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırma ile ilgili bir makaleyi özetledi; hoca sunum sonunda soru sordu ve yorum yaptı. Toplam altı sunum yapıldı.
  - Hoca, sunumların kişinin kendi çalışması değil, bir başkasının çalışmasının özeti olduğunu, bu yüzden 15-20 dakikayı geçmemesi gerektiğini söyledi.
- Sunum 1: Haar ayrık dalgacık dönüşümü (Haar DWT) ile görüntü sıkıştırma (ekrandan, 00:10:00, 00:15:40, 00:18:30, 00:23:10)
  - Dönüşüm, ham sinyalde okunamayan bilgiyi elde etmek için yapılır. En yaygın dönüşüm Fourier dönüşümüdür; dalgacık (wavelet) dönüşümü ise sinyalin hem zaman hem frekans gösterimini verir. Dalgacığın iki temel özelliği ölçek (scale/dilation) ve konumdur (location/translation).
  - Slayttaki iki biçim: sürekli dalgacık dönüşümü $CWT(a,b) = \frac{1}{\sqrt{a}} \int_{-\infty}^{+\infty} f(t)\, \psi^*\left(\frac{t-b}{a}\right) dt$ ve ayrık dalgacık dönüşümü $DWT(m,n) = a_0^{-m/2} \sum_k f(k)\, \psi^*(a_0^{-m}k - nb_0)$.
  - Sıkıştırmada fikir: bir eşik değeri seçilir, büyüklüğü eşiğin altında kalan detay katsayıları sıfırlanır. Daha çok sıfır, daha az yer demektir. Görüntü yaklaşık (approximation) ve detay alt sinyallerine ayrılır.
  - DCT gibi blok tabanlı yöntemler görüntüyü 8x8 ya da 16x16 bloklara böler; dalgacık dönüşümü bütün görüntüye uygulanır, bu yüzden bloklanma bozulması (blocking artifact) olmaz.
  - Haar DWT adımları: çiftlerin ortalamasını al, ortalama ile örnek arasındaki farkı bul, dizinin ilk yarısını ortalamalarla, ikinci yarısını farklarla doldur, normalize et, işlemi ilk yarıya tekrarla. Örnek: $[1,3,5,7] \to [2,6,-1,-1] \to [4,-2,-1,-1]$ (python ile doğrulandı).
  - İki boyutta önce satırlara, sonra sütunlara tek boyutlu filtre uygulanır; LL (yaklaşık görüntü), HL (yatay kenar), LH (dikey kenar), HH (çapraz kenar) alt bantları çıkar. LL üzerinde tekrar edilerek ikinci ve üçüncü seviye elde edilir.
  - Makalenin sonucu: 128x72 görüntü üç seviyede 64x36, 32x18, 16x9'a ayrılmış; eşik 3,6, yalnız 1 yaklaşık katsayı; yeniden oluşturma verimi %99,9865, sıkıştırma skoru %60,8978.
- Sunum 2: Sinir ağlarıyla kayıpsız veri sıkıştırma (ekrandan, 00:28:50, 00:30:00, 00:36:45, 00:40:00)
  - Kodlayıcı her t anında, önceki sembolleri bilerek sinir ağıyla sonraki sembolün olasılık vektörünü hesaplar; sembol aritmetik kodlayıcıyla yaklaşık $-\log_2 p(s_t)$ bitle kodlanır ve model güncellenir. Kod çözücü simetrik çalışır, model parametrelerinin gönderilmesi gerekmez.
  - Ön işlem yoksa her sembol bir byte'tır ($N_s = 256$); ön işlem varsa her sembol bir byte dizisidir ve büyük modellerde yaklaşık 16.000 sembollük bir sözlük kullanılır.
  - Kullanılan modeller LSTM (giriş, unutma ve çıkış kapılı tekrarlayan ağ) ve Transformer'dır.
  - Alt kelime (subword) tabanlı ön işlemci: ilk geçişte büyük harfler kaçış kodlarıyla küçük harfe çevrilir; ikinci geçişte, Byte Pair Encoding'den esinlenerek iyi seçilmiş iki sembol birleştirilip sözlüğe yeni sembol eklenir. Birleştirme, sıfırıncı derece entropideki beklenen azalmaya göre seçilir.

  - enwik8 sonuçları (bpb): gzip -9 2,92; xz -9 1,99; lstm-compress 1,64; CMIX 1,19; LSTM (small) 1,64; Transformer 1,45; LSTM (large1) 1,36; LSTM (large2) 1,34. enwik9'da LSTM (large2) 1,00, CMIX 0,93 bpb.
- Sunum 3: FastText.zip, metin sınıflandırma modellerinin sıkıştırılması (ekrandan, 00:50:00, 00:51:10, 01:00:00)
  - Doğrusal metin sınıflandırıcılar hızlıdır ama kelime ve n-gram gömmelerini sakladıkları için bellek ister. Amaç sınıflandırma doğruluğu ile model boyutu arasındaki uzlaşmadır.
  - PQ (Product Quantization, çarpım niceleme): vektörü, kod çizelgesi denilen önceden tanımlı merkezlerden en yakınına yaklaştırır. $b = 8$ için $2^8 = 256$ merkez vardır. Vektörün normu ve yönü ayrı nicelenirse performans korunur ama vektör başına bir bit fazla gerekir.
  - Kelime dağarcığı budama: en yüksek normlu kelime ve n-gramlar tutulur, ama her belgenin en az bir özelliğinin kalması (kapsama) gözetilir. Hashing hilesi ve Bloom filtresi bellek kazandırır.
  - Sonuç: PQ, gözle görülür performans kaybı olmadan yaklaşık 10 kat sıkıştırma sağlar; doğruluk %0,1-0,5 düşer.
- Sunum 4: RLE, aritmetik kodlama, Punctured Elias ve Goldbach kodlarının karşılaştırılması (ekrandan, 01:10:00, 01:12:35, 01:17:30, 01:20:00, 01:20:40, 01:23:40)
  - Kayıpsız yöntemler (RLE, aritmetik kodlama, Goldbach kodu) metin ve programlar için, kayıplı yöntemler (JPEG, MP3, MPEG) görüntü ve video için kullanılır.
  - RLE: aynı veri d art arda n kez geliyorsa $n@d$ ile değiştirilir. Örnek: AAAAAABBBXXXXXT (15 byte) → 6A 3B 5X 1T (8 byte). Sıkıştırma oranı = sıkıştırılmamış uzunluk / sıkıştırılmış uzunluk.
  - Punctured Elias kodu (Peter Fenwick): 13 = 1101, ters çevrilince 1011; 0001 bayrakları önüne eklenir, en sağdaki bit atılır: 0001|101.
  - Goldbach kodları "2'den büyük her çift sayı iki asal sayının toplamıdır" sanısına dayanır. G0: n, $2(n+3)$ çift sayısına çevrilir ve asal toplamı ters yazılır. G1: $n = P_i + P_j$ ($i \leq j$) bulunur ve $(i, j - i + 1)$ çifti gamma kodlarıyla yazılır. G2 G1'in uzantısıdır.
  - Deney sonuçları (bit): 40 bit girdi için RLE 80, aritmetik 32, Punctured Elias 64, Goldbach 14; 80 bit için 64, 56, 75, 28; 120 bit için 240, 65, 88, 49.
- Sunum 5: Doğal dil işleme için CNN sıkıştırma: niceleme ve budama (ekrandan, 01:30:00, 01:31:30, 01:40:00)
  - Amaç, FPGA üzerinde gömülü sinir ağı hesaplamalarında niceleme (quantization) ve budamanın (pruning) etkisini incelemektir. Niceleme karmaşıklığı, budama işlem sayısını azaltır.
  - Kayan nokta standardında tek duyarlıkta (32 bit) 1 işaret, 8 üs, 23 mantis biti vardır. Dinamik sabit noktada $frac_bits = total_bits - int_bits - 1$; örneğin 16 bitte 10 kesir biti, 5 tamsayı biti, 1 işaret biti.
  - Budamada küçük ağırlıklar (örnekte 0,02 eşiği) atılır, çünkü toplam hesaba etkileri azdır.
  - Model Yoon Kim'in CNN-static modelidir: kelime gömme, 2 evrişim ve 2 yoğun katman. MR, SST-1, SST-2, Subj, TREC, CR ve MPQA veri setleri kullanılmıştır.
  - Doğruluk belirli bir kaybı aşmadan her katmanın ağırlık ve aktivasyonları için en düşük bit sayısı aranmıştır (örneğin gömme için 5 bit). Budama eşiği arttıkça ve bit sayısı azaldıkça hesap oranı ve doğruluk düşer.
- Sunum 6: Dinamik değişkenden değişkene sıkıştırma (D-v2v) (ekrandan, 01:50:00, 01:52:20, 01:55:30, 02:00:00)
  - Kelime tabanlı, tek geçişli, dinamik bir sıkıştırıcıdır; Dynamic End-Tagged Dense Code (DETDC) temel alınmıştır. Ayrıştırma, kodlama ve kod çözme adımlarından oluşur; kodlayıcı ve kod çözücü senkron olmalıdır.
  - Terminal sembol tek kelimedir ve yeni kelime gelince oluşturulur. Terminal olmayan sembol en az iki sembolden oluşur. Yeni bir sembol gönderilince, önceki gönderilen sembolle birleşimi de listeye eklenir.
  - Örnek mesaj "the more I know about you the more I know about me": "you"ya gelindiğinde liste S0 "the", S1 "more", S2 "the more", S3 "I", S4 "more I", S5 "know", S6 "I know", S7 "about", S8 "know about", S9 "you", S10 "about you" olur. Sonra "the more" okunur, "the more I" listede olmadığı için S2 gönderilir ve S11 = S9||S2 "you the more" eklenir.

- Kod çizelgesi her sembol için $\langle l, r, freq, voc \rangle$ tutar. Terminal sembolde l kelimenin kendisi, $r = -1$'dir; terminal olmayanda l ve r bileşenlerin indisleridir. Sık kullanılan sembole kısa kod verilir; sıralama pos ve top dizileriyle tutulur.
- Sıkıştırma oranı tablosunda değer, sıkıştırılmış boyutun yüzdesidir (küçük olan iyidir). D-v2v, DETDC'yi normal İngilizce metinlerde 10 puandan fazla geçmiş; p7zip ve lzma'ya genelde yetişememiştir. Sıkıştırmada bellek kullanımı yüksektir.

Hocanın Özellikle Vurguladığı Kısımlar

- Dalgacık dönüşümünün başarısı
  - Hocaya göre dalgacık dönüşümünün arkasında çok matematik vardır, ama genelde hazır fonksiyonlar (özellikle Matlab'de) kullanılır.
  - Dalgacık dönüşümünün DCT'den daha iyi çalışması gerekir; hoca kendi kullanımında daha yüksek başarı aldığını söyledi. Makaledeki %60'lık skoru düşük buldu. Yalnız 1 yaklaşık katsayı kullanılmasını ve ayrıştırmanın 3. seviyede bırakılmasını (hiyerarşi derinliğini) önemli etkenler olarak gösterdi.
- Okumak ile anlamak arasındaki fark
  - Hoca, makaleyi tercüme edip aktarmanın yetmediğini, sunumun birkaç cümleyle özetlenebilmesi gerektiğini söyledi. Makale seçmenin çok önemli olduğunu vurguladı.
- Kayıplı mı, kayıpsız mı?
  - FastText.zip sunumunda hoca, sık kelimeler atılıyorsa geri dönüşün nasıl olacağını, bunun kayıplı sıkıştırma olup olmadığını sordu.
- Sıkıştırma oranının okunması
  - Tabloda %35 yazıyorsa bu, 100 birimin 35 birime düştüğü, yani %65'lik bir kazanç olduğu anlamına gelir. Değer performans değil, orandır.
- Elias ve Goldbach kodları
  - Hoca, Elias kodlarının ilk haftalarda anlatıldığını, Goldbach kodlarının ise derste verilmediğini söyledi.

Kısa Tekrar Notları

- Dalgacık dönüşümü zaman ve frekans bilgisini birlikte verir; bütün görüntüye uygulandığı için bloklanma bozulması yoktur.
- Haar DWT: ortalama ve fark; $[1, 3, 5, 7] \to [2, 6, -1, -1] \to [4, -2, -1, -1]$.
- 2B DWT: önce satır, sonra sütun; LL yaklaşık görüntüdür, HL/LH/HH kenar bilgisidir; LL tekrar ayrıştırılır.
- Sinir ağı tabanlı kayıpsız sıkıştırma: model olasılığı tahmin eder, aritmetik kodlayıcı $-\log_2 p$ bit harcar, kod çözücü aynı modeli kurar.
- enwik8'de büyük LSTM 1,34 bpb ile gzip (2,92) ve xz'den (1,99) iyidir, ama çok yavaştır.
- PQ = Product Quantization; $b = 8$ için 256 merkez; FastText.zip yaklaşık 10 kat sıkıştırma ve %0,1-0,5 doğruluk kaybı verir.
- RLE: $n@d$; AAAAAABBBXXXXXT (15 byte) → 6A3B5X1T (8 byte).
- Punctured Elias: 13 → 0001|101.
- Goldbach: G0 ($2(n+3)$), G1 ($(i, j-i+1)$ gamma kodları), G2 (G1'in uzantısı).
- CNN sıkıştırma: niceleme bit sayısını, budama işlem sayısını azaltır; $frac_bits = total_bits - int_bits - 1$.
- D-v2v: kelime tabanlı, terminal ve terminal olmayan semboller, $\langle l, r, freq, voc \rangle$ kod çizelgesi, pos ve top dizileri.
- Sıkıştırma oranı %35 → %65 kazanç.

Detaylı Açıklamalar

- Haar DWT örneği (ekrandan, 00:15:40)
  - Sinyal $[1, 3, 5, 7]$ olsun. İlk iterasyonda çiftlerin ortalaması alınır: $(1+3)/2 = 2$, $(5+7)/2 = 6$. Farklar örnek eksi ortalama olarak bulunur: $1 - 2 = -1$, $5 - 6 = -1$. Dizinin ilk yarısı ortalamalarla, ikinci yarısı farklarla doldurulur: $[2, 6, -1, -1]$.
  - İkinci iterasyonda yalnız ilk yarı işlenir: $(2 + 6)/2 = 4$, $2 - 4 = -2$. Sonuç

$[4, -2, -1, -1]$ olur. Slaytta normalize adımı listede yer alır, ama örnekte uygulanmamıştır.
  - İlk değer sinyalin ortalamasıdır, geri kalanlar detaydır. Detay katsayıları küçükse eşikle sıfırlanabilir; bu sıkıştırmayı sağlar.
- Görüntüde dalgacık ayrıştırması (ekrandan, 00:18:30 ve 00:20:00)
  - Satırlara uygulanan DWT görüntüyü L ve H olarak ikiye böler; sütunlara uygulanan DWT ile LL, HL, LH, HH elde edilir. LL görüntünün kaba (küçültülmüş) sürümüdür.
  - Slaytta 512x512 bir görüntünün LL bandının art arda ayrıştırılmasıyla 256x256, 128x128, 64x64 ve 32x32 boyutlu yaklaşık görüntüler gösterildi. Bir milyon katsayının yalnız birkaç binlik kısmı büyüktür; küçükler sıfırlanınca görüntü kalitesi pek değişmez.
- Sinir ağlarıyla kayıpsız sıkıştırma ve ön işlem
  - İki tür ön işlem anlatıldı: küçük LSTM modeli için CMIX ön işlemcisi (kelimeleri önceden hazırlanmış bir sözlükten değiştirir) ve büyük modeller için alt kelime tabanlı ön işlemci.
  - Sunan öğrenciye göre makalede Transformer modeli LSTM'den daha kötü sonuç vermiştir. enwik8 tablosunda sıkıştırma hızı LSTM (small) için 41,7 kB/s, Transformer için 1,79 kB/s, gzip için 17.400 kB/s'dir; yani sinir ağı modelleri çok daha iyi oran verir ama çok yavaştır.
- FastText.zip ve PQ
  - Sunumda sınıflandırma modeli $\sum_{n=1}^{N} \ell(y_n, BAx_n)$ kaybını en aza indirir; x_n tek-sıcak vektörlerden oluşan bir torba, y_n n'inci belgenin etiketidir. Büyük kelime haznesi ve çıktı alanında A ve B matrisleri gigabaytlarca bellek gerektirebilir (ekrandan, 00:50:00).
  - Bir hashtag tahmin görevinde 312.116 etiket ve yaklaşık 1,5 milyon kelimelik sözlük kullanılmıştır. Tam model eksik kelimeler yüzünden test setinin %11,6'sını kaçırır; 1 milyon özellikle saf budama %30-40'ını kaçırır; kapsamayı gözeten budama ise kapsama kaybı yaşamaz.
  - Sonuç slaydına göre yaklaşım, karşılaştırma ölçütlerinde daha az bellekle biraz daha düşük doğruluk verir (ekrandan, 01:00:00).
- Goldbach kodlarının özel durumları (ekrandan, 01:20:40)
  - 1 ve 2 tamsayıları 110 ve 111 olarak kodlanır; başka hiçbir kod 11 ile başlamaz.
  - Çift sayılar G1'deki gibi kodlanır, ama $(i, j - i + 1)$ yerine $(i + 1, j - i + 1)$ çifti kullanılır. $i = 1$ ise 2'nin gamma kodu olan 010 yazılır. Böylece çift sayıların kodu 1 ile başlamaz.
  - n bir P_i asalıysa $(i + 1)$'in gamma kodu ve ardından tek bir 1 yazılır.
  - n tek ise kod tek bir 1 ile başlar, ardından $n - 1$ çift sayısının G2 kodu gelir.
- D-v2v'de frekans güncellemesi (ekrandan, 01:55:30)
  - "you" eklendiğindeki durumda frekansı 1 olan semboller pos dizisinin 0-5 konumlarında, frekansı 0 olanlar 6'dan itibaren durur; top[0] = 6, top[1] = 0'dır.
  - Sonraki adımda S2 "the more" kodlanıp gönderilir. Kod çizelgesinin 2. indisinin frekansı 1 artırılır. Frekansı 0 olanların başlangıcı 6 olduğu için voc[2] ile voc[6] yer değiştirir; pos[6] = 2, pos[9] = 6 ve top[0] = 7 olur.
  - Kod çözücü yalnız kod çizelgesi (offset, uzunluk, frekans) ve yardımcı tabloyla çalışır. Ya bilinen bir sembolün kodu ya da kaçış (escape) kodu ve ardından düz kelime gelir. Senkron kalmak için son iki çözülen sembolden yeni bir terminal olmayan kayıt eklenir.
  - Süre açısından D-v2v p7zip, lzma ve Re-Pair'den hızlı, v2vdc ile aynı düzeyde, bzip2'den yavaştır. Sıkıştırmada çok sayıda yapı kullandığı için bellek ihtiyacı yüksektir; gelecek çalışma olarak bellek ihtiyacının azaltılması planlanmıştır.
- Soru: D-v2v sözlük tabanlı bir sıkıştırmaya dönüştürülebilir mi?
  - Cevap: mümkündür, çünkü yöntem zaten kelime bazlı çalışır (derste sözlü, 02:07:19).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.